CS5290: Memory-Centric Computing Architectures

A1 Slot
Tuesday – 4 pm to 4:55 pm
Wednesday – 3 pm to 3:55 pm
Thursday – 2 pm to 2:55 pm
Monday – 5 pm to 5:55 pm (Reserved)
Room 5405

Motivation

- Why this course?
  - Memory wall limits modern computing performance
  - AI workloads are increasingly data-intensive
  - Data movement dominates latency and energy
  - Emerging memory-centric architectures address these challenges

Course Objectives

- ❑ Understand the memory wall and data movement challenges
- ❑ Learn Near-Memory and In-Memory Computing
- ❑ Study ReRAM-based computing architectures
- ❑ Explore reliability and hardware security
- ❑ Introduce neuromorphic computing architectures

Course Syllabus

Module	Topic	Lecture Hours***
1	Computer Architecture Foundations	4
2	Memory Wall & Data Movement	5
3	AI Workloads & Accelerator Architectures	5
4	Near-Memory Computing	7
5	In-Memory Computing & ReRAM	8
6	Reliability, Faults & Security	7
7	Neuromorphic Computing	6

***** Tentative**

References

Hennessy & Patterson, *Computer Architecture: A Quantitative Approach*, 5th Ed.
Liu, Han & Lombardi (Eds.), *Design and Applications of Emerging Computer Systems*, Springer, 2024

Assessments

Assessment	Marks	Weightage
Assignment-1	20	10%
Quiz-1	20	10%
Mid-Sem	60	20%
Quiz-2	20	10%
Project	40	20%
End-Sem	90	30%

Teaching Assistants (TAs)

- EVANSTAR FIELD WAR (246101006)
- BANTEILANG MUKHIM (256101006)
- RISHABH CHAMOLI (254101047)

What is Computer Architecture?

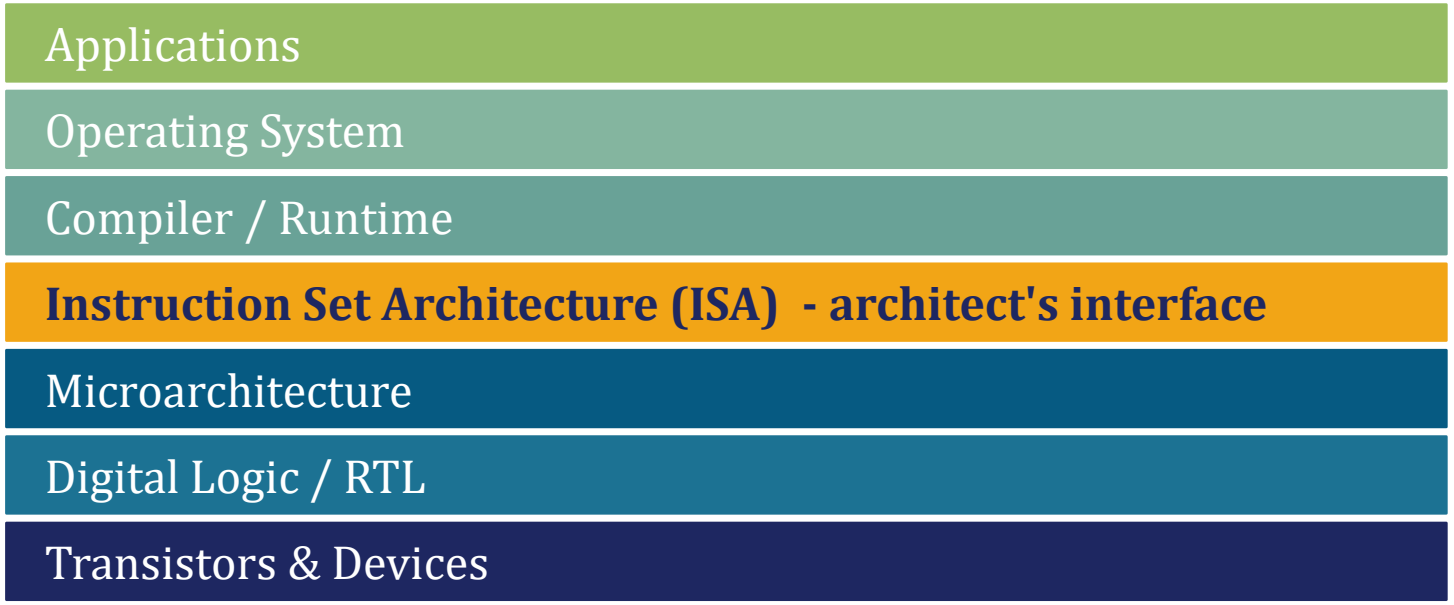
- Computer architecture is the discipline that bridges the software a programmer writes and the physical hardware that executes it.
- The architect's job is to design machines that run programs correctly, quickly, and efficiently while navigating constraints of power, cost, area, and reliability.
- Important factors while designing a system:
  - CORRECTNESS - Faithfully implement the ISA contract so every program behaves as defined.
  - PERFORMANCE - Maximize useful work per unit time using parallelism, locality, and pipelining.
  - EFFICIENCY - Deliver that performance within strict power, thermal, and cost budgets.

Three Layers the Architect Designs

- ❑THE CONTRACT - Instruction Set Architecture (ISA)
  - The set of instructions a CPU promises to support, plus its register file and rules for accessing memory. The ISA is the visible interface to software.
  - Examples: RISC-V, ARM, x86-64.
- ❑THE IMPLEMENTATION - Microarchitecture
  - How a particular chip realizes the ISA inside: pipeline organization, caches, branch predictors, ALUs, issue width, and out-of-order machinery.
  - Two chips can share an ISA yet differ vastly in microarchitecture.
- ❑THE PHYSICS - Hardware
  - Logic gates, transistors, layout, and clock distribution. This layer fixes how fast the chip can switch, how much it dissipates, and how hot it gets.
  - Process node, voltage, and floorplan live here.

Levels of Abstraction

- A modern computer is built as a stack of layers. Each layer exposes a clean interface upward while hiding the messy details below.
- Architects work at the boundary where software meets silicon: between the ISA and the hardware that implements it.



WHY LAYERS HELP

- Each layer can evolve independently while the interface above stays stable.
- A compiler writer need not know transistor physics.
- A circuit designer need not know the source language.
- The ISA is the most durable boundary, it long-outlives any one chip.

RISC-V - A Modern, Open ISA

- RISC-V is a free, open-standard ISA designed at UC Berkeley. It is simple, regular, and modular; contains every idea we need to understand modern processors.

32 GENERAL-PURPOSE REGISTERS

- Registers x0–x31, each 32 bits wide. x0 is hardwired to zero.
- All arithmetic operates on register values, not memory.

LOAD–STORE ARCHITECTURE

- Memory is touched only by lw (load) and sw (store).
- Every other instruction reads and writes registers, a hallmark of RISC design.

FIXED 32-BIT INSTRUCTIONS

- Every instruction is exactly 4 bytes.
- Fetch and decode become simple, predictable, and pipeline-friendly.

THREE-OPERAND FORMAT

- Most instructions take two source registers and one destination.
- e.g. add x5, x6, x7 computes x5 ← x6 + x7.

RISC-V Instruction Formats

- Every 32-bit RISC-V instruction is packed into a fixed layout of fields.
- There are six format types in total, the popular ones are: R-type and I-type

R-type - register/register arithmetic

funct7	rs2	rs1	funct3	rd	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

Example: add x5, x6, x7 → rd=5, rs1=6, rs2=7, funct3=0, funct7=0, opcode=0110011

I-type - immediates and loads

imm[11:0]	rs1	funct3	rd	opcode
12 bits	5 bits	3 bits	5 bits	7 bits

Example: addi x5, x6, 12 or lw x5, 0(x6)

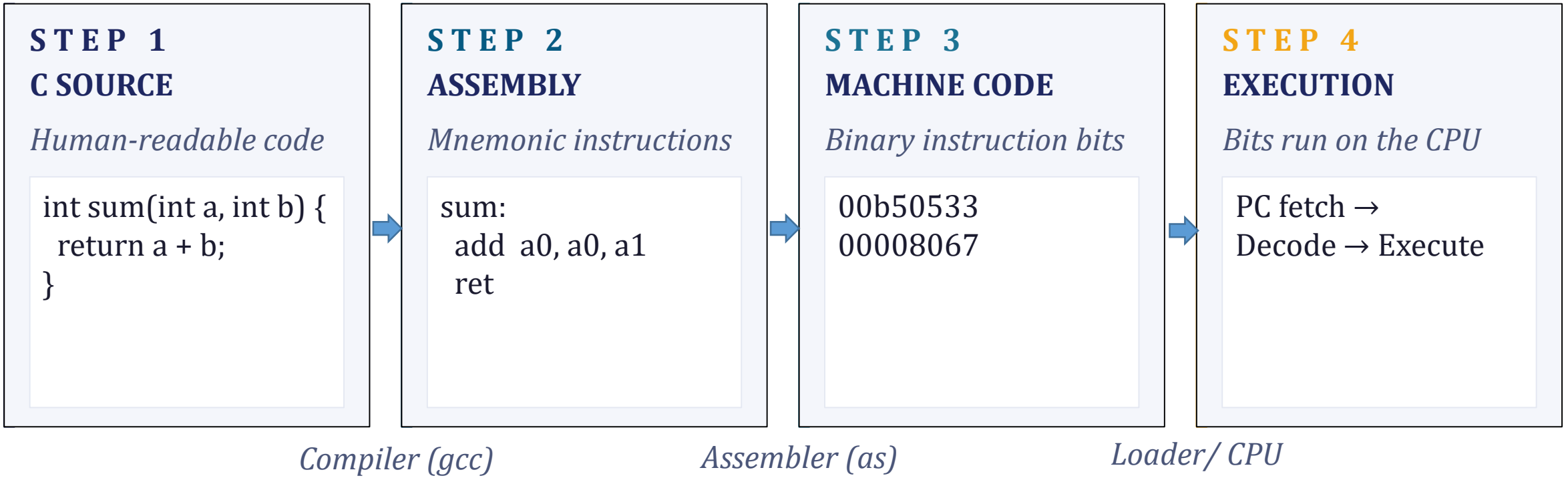
Both formats put rd, rs1, and opcode at fixed positions, so decode can begin before the rest of the instruction is even parsed.

Why an Open ISA Matters

- Historically, every commercially relevant ISA was proprietary.
- RISC-V changed the landscape by offering a royalty-free, extensible base ISA.
- The result is an academic and industrial ecosystem where new ideas can be prototyped, taught, and shipped without licensing friction.
- RISC-V ISA is:
  - Modular by design
  - Education-friendly
  - Industrial adoption
  - Research catalyst

From C to Machine Code

- Before a processor can execute your program, it must be translated from human-readable source code down to the binary that hardware actually understands.
- Three tools, used in sequence, perform this translation.



The Stored-Program Concept

- In a stored-program computer, instructions and data live in the same memory and are indistinguishable at the bit level.
- The CPU uses a Program Counter (PC) to fetch instructions one after another, executes each, then advances to the next.
- This 1945 idea, attributed to von Neumann and the EDVAC team, underpins every general-purpose processor built since.

MEMORY

0x1000	**0x00b50533**	*add x10, x10, x11*
0x1004	**0x00008067**	*ret*
0x1008	**0x0000002A**	*data: 42*
0x100C	**0x00000007**	*data: 7*

THE FETCH–EXECUTE LOOP

- Fetch the instruction at memory[PC].
- Decode the bits to determine the operation.
- Execute the operation in the ALU or memory unit.
- Update registers and/or memory with the result.
- Advance PC to the next instruction and repeat.

The Datapath

- The datapath is the physical highway of wires and functional blocks that carries an instruction from fetch to write-back.
- Every instruction visits some subset of these five units. The control unit decides what each unit does each cycle.

IF	ID	EX	MEM	WB
Instruction Fetch	**Instruction Decode**	**Execute**	**Memory Access**	**Write-Back**
Read instruction at memory[PC]	Read registers, generate immediate	ALU operation or branch test	Loads and stores	Write result to register file

- The control unit issues signals each cycle (e.g. RegWrite, ALUSrc, MemRead) that steer data through these blocks.

Thank you